```
  else
    @project = @workflow.project
    render :new
  end
end
```

The controller largely defers to a workflow object that does the actual work:

mocks/01/app/workflows/creates_project.rb

```
class CreatesProject
  attr_accessor :name, :project, :task_string

  def initialize(name: "", task_string: "")
    @name = name
    @task_string = task_string || ""
    @success = false
  end

  def success?
    @success
  end

  def build
    self.project = Project.new(name: name)
    project.tasks = convert_string_to_tasks
    project
  end

  def create
    build
    result = project.save
    @success = result
  end

  def convert_string_to_tasks
    task_string.split("\n").map do |one_task|
      title, size_string = one_task.split(":")
      Task.new(title: title, size: size_as_integer(size_string))
    end
  end

  def size_as_integer(size_string)
    return 1 if size_string.blank?
    [size_string.to_i, 1].max
  end
end
```

Now you have two different pieces of point-of-failure logic to deal with:

- In the event of a database failure, the workflow object should behave gracefully and return the fact that the creation did not succeed.
- In the event the workflow doesn't succeed, the controller redirects to redisplay the form.